

Hotline Networking

Eric Grosse grosse@gmail.com

version 2026-05-06 latest version will be at github.com/n2vi/hotline

This draft has only minor changes since the 2020 version.

This document makes a concrete proposal for the network link between the Puck and the Broker hotline devices owned by one country. We seek a small, easily reviewed network stack with minimal OS support. It should depend on little in the surrounding office environment.

In the CATALINK architecture, there is a small device called the Puck owned by a hotline Principal that holds all secrets for their end-to-end encrypted hotline. It may only be intermittently powered on, so relies for connectivity on their Broker, a well-connected and always-on relay server with somewhat less security sensitivity. Brokers talk to other principals' Brokers using a highly resilient network called ROCCS, which might include a mesh RF network and satellites and the Internet. This document sketches how Puck talks to Broker.

Puck is betting on radically reduced attack surface compared to other computers, so is expected to run a small kernel and a handful of userland software for a stack that should be reviewable by people from many backgrounds. A typical TCP/IP network stack (and even more, a typical video driver and windowing system) would be larger than the entire rest of the kernel so the CATALINK team identified early on that a simpler networking alternative is a crucial element of technical credibility for our project. This is only feasible because Puck only needs to interoperate with Broker, and hence can drop all the legacy compatibility constraints that defeat most ambitious security initiatives.

Broker runs a larger operating system such as Linux or OpenBSD and, besides talking to Puck, must be capable of handling many other communication channels such as Internet, private fiber, satellite links, and HF radio. We will endeavor to make it secure but recognize that with the necessary added complexity comes added risk. So Broker holds mild secrets, enough to deter most wiretapping or impersonation, but nothing crucial to hotline message content.

In CATALINK we believe we can achieve better simplicity because we are not trying for ultimate performance. Hardware has gotten orders of magnitude faster since the '60s era Moscow-Washington hotline that we're reconstructing and we're willing to spend some of that on simplicity. Not that performance doesn't matter; our final system needs to feel snappy.

Puck and Broker might live in the same office building and be connected by CAT5 cable through a Gigabit Ethernet switch. We assume a low bandwidth-delay product and a reasonably uncongested network. We do not entirely trust the network, though we are willing to assume that adversaries would find it challenging to shut it down. More specifically, from a security threat perspective, we feel the need for encryption to protect against wiretap or forgery but do not feel the need for extraordinary defense against Denial-of-Service packet flooding or against physical disconnection. To the extent feasible, we would like any network outage to be diagnosable by relatively unskilled staff, who can spot unplugged or frayed cables or power failures, but not DNS

misconfiguration. Initial setup may require an expert, as would ongoing systems administration of the Broker.

The [Precursor](#) device uses WiFi rather than wired ethernet but the security relevant network stack is a COM bus that is also designed for simplicity so might be ok from infosec perspective, though to hide location of the Principal, WiFi is likely unacceptable.

It is likely that an implementation will deploy multiple Brokers (think one in the White House and one on Air Force One) exchanging state frequently. For messages to get through it would be enough that the Puck be able to talk to any one of these Brokers. For initial clarity, assume a single Broker.

The current state of the proposal is aimed at prototypes on common platforms but prepares for a tiny OS on future devices, and therefore we imagine a primitive UDP interface provided by the operating system and limit what we ask of it to handling the network hardware and multiplexing packets (based on UDP port number) to and from independently scheduled user-mode applications which are responsible for their own retransmission and encryption. It would have been enough to just send raw jumbo ethernet frames from Puck to Broker but that seemed likely to lead to portability problems, so we've postponed exploring that just yet. We do not ask for DNS or routing configuration or the rest of a normal IP infrastructure.

To get reliable UDP, we're using a Go-Back-N protocol which can be implemented with a circular buffer of packets, integrated acks, and uint32 packet sequence numbers preserved across invocations. (For historical origins, see [Plan9 rudp](#).)

puckfs

Instead of general message passing, we conjecture all we need is remote procedure call and in fact we think we can get by with a remote filesystem, reading and writing full files. For simplicity, we decided:

- no walk, always use canonicalized pathname
- no stat, that info comes when reading directory
- no mkdir; directories are created as needed when file is written
- no symlinks, no hard links, no locking, no ACLs

The **client** calls

```
p, err = puckfs.Dial(secretFile)
err = p.Close()
data, err = p.ReadFile(path)
err = p.WriteFile(path, data)
err = p.Remove(path)
fi, err = p.ReadDir(path)
```

Here `secretFile` is a local file containing JSON fields: `RemoteAddr`, `MTU`, `KeyID`, `Secret`. `KeyID` is unique to a (client,server) pair and is used in both directions. Calls are synchronous.

The filesystem is single-user, so has no permissions or access control beyond the authenticated encryption of the network link.

The **server** process supports a single remote user and runs as a custom local userid with access to just the one dedicated directory tree.

If we need more functionality, we'll try files first. For printing, don't implement CUPS but WriteFile a PDF from the puck to a spool directory on Broker. It seems highly likely that the principal creating and responding to messages on the Puck may wish to include text and photos from staff. Again the puckfs mechanism may be helpful via relay to legacy network file systems.

FAQ

Q: Oh my goodness, why all this invention when you could just use Signal?

A: Good question. The nuclear command authorities likely already have and use Signal, including the video call features. Certainly I use it with my own friends and family, I send financial support to Signal, and I respect their cryptography that is much more sophisticated than this little hotline. Two reasons: 1) we're trying to build a deep backup here that works when DNS and AWS and lots else has failed; 2) we're trying to build something simple enough to be audited or completely re-implemented by multiple adversaries and yet still be interoperable. For good reasons, Signal does not allow that.

Q: What is the user interface?

A: Since Puck hardware is still to be settled, discussion of UI is somewhat premature. For this proof-of-concept implementation, we're building on top of OpenBSD and running commands within the acme text editor. Error messages are seen directly on that screen. I'm looking forward to an implementation soon on Precursor devices. There room for choice.

TODO

We must provide a safe way to set (and reset) the shared secret, both for puck-broker and puck-puck. We'll probably offer a standalone implementation of authenticated key establishment using ML-KEM, as shown in Figure 11 at the end of NIST SP 800-227.

We would like a **notification** channel, for example to be alerted when a directory changes.

Implicitly, the intent here is to standardize the wire format of puckfs as well as the client and server API. It was tempting to adopt 9P2000, but that seemed overkill for our needs. Such a standard makes it easy to later adopt different languages for implementing Puck and Broker; indeed, we anticipate our hotline code will soon be ported to Rust.

[AdamWick] Four things you might want to prove:

1. That the designed puck-to-puck crypto protocol maintains the integrity and confidentiality of the data. Since you're not actually trusting the puck-to-broker or broker-to-broker channels to maintain C/I, proving stuff about them isn't nearly as important, but puck-to-puck needs to be proven. State of the art there is automated theorem proving, Coq, Isabelle, and so on.
2. That the puck-to-broker/broker-to-puck protocol doesn't have any weird corners; more formally, you probably want a couple "always eventually" properties, like "the message from the puck always eventually makes it to the broker if it is physically possible." SPIN is still around for this sort of thing. I've also heard of people using Ivy, TLA+, P, and Alloy these days. Some of the newer tools can also generate test vectors to test your implementation, too. (Although, of course, it'd be lovely to verify it, too.)
3. That the broker-to-broker protocol similarly doesn't go into a corner. Same as #2, perhaps, but it depends on how you design things. Again, you're probably looking to prove properties around "at most once" delivery, since you can use your monotonic counter to clear double-deliveries. Also, if the broker-to-broker protocol goes full mesh (which, given the context, is probably wise), those protocols are often hard to reason about in a dynamic universe. Even if I pulled something off the shelf, I'd want to get a good formal grounding for it given whatever fundamental technologies are used. Dynamic routing across the world is hard!
4. That the implementation of the puck's crypto and messaging actually implement the design you came up with in #1. For this, I will recommend Galois's SAW and Crux tools.[/AdamWick]

I thank Dave Presotto for comments on this proposal and even more for teaching me all this in the Plan 9 days. I thank Alexa Wehsener and Leah Walker for clarifying that ROCCS refers to the network; Broker is the proxy device. Adam Wick added some great suggestions on which formal methods may be most relevant.